# JavaScript Code

```
const { test, expect } = require('@playwright/test');
const axios = require('axios');
const nodemailer = require('nodemailer');
const fs = require('fs');
const PizZip = require('pizzip');
const Docxtemplater = require('docxtemplater');

require('dotenv').config();

const { roles, skills } = require('../resume.json');

const transporter = nodemailer.createTransport({
    service: 'gmail',
    auth: {
        user: process.env.EMAIL,
        pass: process.env.EMAIL_PASS
    }
});

const candidate = {
    fullName: 'Vaishak Upadhyaya',
    email: 'testmailauto6096@gmail.com',
    phone: '741 6350328',
    linkedin: 'https://www.linkedin.com/in/vaishak-upadhyaya-771596420/',
    location: 'Hyd, TG 500068',
    relocation: 'Yes',

    // Replace these test values with the actual candidate's details
    workAuthorization: 'STEM OPT',
    availability: 'Immediate',
    experience: '7+ Years',
    expectedSalary: 'C2C only (discuss with employer)'
};

test.use({
    storageState: 'tests/login.json'
});

test('LinkedIn Search', async ({ page }) => {

    test.setTimeout(180000);
    const context = page.context();

await context.grantPermissions(
    ["clipboard-read", "clipboard-write"],
    { origin: "https://www.linkedin.com" }
);
    // Open LinkedIn
    await page.goto('https://www.linkedin.com/feed/');
    await expect(page).toHaveURL(/feed/);
```

```
51
52      // Search
53      await page.getByTestId('typeahead-input').click();
54
55      await page
56          .getByTestId('typeahead-input')
57          .fill('"JAVA DEVELOPER" C2C -fulltime -w2 -hotlist -bench -sales "United
    States"');
58
59      await page.goto(
60          'https://www.linkedin.com/search/results/all/?
    keywords=%22JAVA%20DEVELOPER%22%20C2C%20%22United%20states%22%20-fulltime%20-w2%20-
    hotlist%20-bench%20-sales&origin=HISTORY&position=0',
61          {
62              waitUntil: 'commit'
63          }
64      );
65
66      // Filter by posts
67      await page
68          .getByRole('radio', { name: 'Filter by Posts' })
69          .click();
70
71      // Filter past 24 hours
72      await page
73          .getByRole('button', { name: 'Filter by Date posted' })
74          .click();
75
76      await page
77          .getByRole('radio', { name: 'Past 24 hours' })
78          .click();
79
80      await page
81          .getByRole('link', { name: 'Show results' })
82          .click();
83          const recruiter = recruiterName.trim();
84
85  const keyword =
86      foundRoles.length > 0
87          ? foundRoles[0]
88          : role;
89
90  const query = `site:linkedin.com "${recruiter}" "${keyword}"`;
91
92  console.log("SerpAPI Query:", query);
93          const response = await axios.get(
94      "https://serpapi.com/search.json",
95      {
96          params: {
97              engine: "google",
98              q: `site:linkedin.com "${recruiter}" "${keyword}"`,
99              api_key: process.env.SERP_API_KEY
100         }
101     }
```

```
);
let postUrl = "Not available";

if (
    serpResponse.data.organic_results &&
    serpResponse.data.organic_results.length > 0
) {
    postUrl = serpResponse.data.organic_results[0].link;
}

console.log("Post URL:", postUrl);
console.log(JSON.stringify(response.data, null, 2));
    // Find recruiter profile links
    const profileLinks = page.locator('a[href*="/in/"]');

    await profileLinks
        .first()
        .waitFor({ state: 'attached' });

    const hrefs = await profileLinks.evaluateAll(links =>
        links.map(link => link.href)
    );

    const uniqueProfiles = [...new Set(hrefs)].slice(0, 2);

    console.log(
        'Unique profiles found:',
        uniqueProfiles.length
    );
    for (const profileUrl of uniqueProfiles) {

        console.log('\n------------------------------');
        console.log('Profile URL:', profileUrl);
const profileMatches = page.locator(`a[href="${profileUrl}"]`);

let profileLink = null;
let recruiterName = "";

const totalMatches = await profileMatches.count();

for (let i = 0; i < totalMatches; i++) {

    const link = profileMatches.nth(i);

    const text = ((await link.textContent()) || "").trim();

    if (text.length > 0) {

        profileLink = link;

        recruiterName = text
            .replace(/Verified Profile/gi, "")
            .replace(/•\s*\d+(st|nd|rd|th)\+?/gi, "")
            .replace(/\s+/g, " ")
```

```
            .trim();

        break;
    }
}

if (!profileLink) {

    console.log("Recruiter name not found.");

    profileLink = profileMatches.first();
}

console.log("Recruiter name:", recruiterName || "Not found");

const post = profileLink.locator(
    'xpath=ancestor::*[@role="listitem"][1]'
);
const allLinks = post.locator("a");
const count = await allLinks.count();

console.log("Links inside post:");

for (let i = 0; i < count; i++) {
    const href = await allLinks.nth(i).getAttribute("href");
    console.log(i, href);
}
        let jobDescription =
            await post.innerText();
        const text =
            jobDescription.toLowerCase();
        const removeWords = [
    "Follow",
    "Like",
    "Comment",
    "Repost",
    "Send",
    "Share",
    "See more"
];

jobDescription = jobDescription
    .split("\n")
    .filter(line => {
        line = line.trim();

        if (!line) return false;

        if (removeWords.includes(line))
            return false;

        if (/^\d+\s*(m|min|h|hr|hrs|d|day|days)\s*•?$/i.test(line))
            return false;
        if (/^(•\s*)?\d+(st|nd|rd|th)\+?$/i.test(line))
```

```
            return false;
        return true;
    })
    .join("\n");
    const timestampLink = post.locator(
    'a[href*="/posts/"], a[href*="/feed/update/"], a[href*="/activity-"]'
).last();


        // ---------------------------------
        // Detect roles + skills
        // ---------------------------------

        const foundSkills = skills.filter(skill =>
            text.includes(
                skill.toLowerCase()
            )
        );

        const foundRoles = roles.filter(role =>
            text.includes(
                role.toLowerCase()
            )
        );

        console.log(
            'Roles found:',
            foundRoles
        );

        console.log(
            'Skills found:',
            foundSkills
        );

        const role =
            foundRoles.length > 0
                ? foundRoles[0]
                : 'Java Developer';

        // ---------------------------------
        // Generate tailored resume
        // ---------------------------------

        const content = fs.readFileSync(
            './Resume.docx',
            'binary'
        );

        const zip =
            new PizZip(content);

        const doc =
            new Docxtemplater(zip, {
```

```
                paragraphLoop: true,
                linebreaks: true
            });

        doc.render({
            roles: foundRoles.join(', '),
            skills: foundSkills.join(', ')
        });

        const buffer =
            doc.getZip().generate({
                type: 'nodebuffer'
            });

        fs.writeFileSync(
            './MyResume.docx',
            buffer
        );

        console.log(
            'Tailored resume generated'
        );
        const mail = post
            .locator('a[href^="mailto:"]')
            .first();
        let email;

        if (await mail.count() > 0) {

            const mailto =
                await mail.getAttribute('href');

            if (mailto) {
                email = mailto
                    .replace('mailto:', '')
                    .split('?')[0]
                    .trim();
            }

            console.log(
                'Email found directly:',
                email
            );

        } else {

            console.log(
                'No direct email. Using SignalHire...'
            );

            const response =
                await axios.post(
                    'https://www.signalhire.com/api/v1/candidate/search',
                    {
```

```
                        items: [profileUrl],
                        withoutWaterfall: true
                    },
                    {
                        headers: {
                            apikey:
                                process.env.API_KEY
                        }
                    }
                );

            console.log(
                JSON.stringify(
                    response.data,
                    null,
                    2
                )
            );

            email = response.data.email;
        }
        const emailBody = `Dear ${recruiterName},

I hope this email finds you well.

I saw your recent hiring post on LinkedIn regarding a ${role} opportunity and would like to submit my application for this role. I have attached my resume to this email for your review.

Here is a summary of my profile for your quick review:
- Candidate Full Name: ${candidate.fullName}
- Email Address: ${candidate.email}
- Phone Number: ${candidate.phone}
- LinkedIn Profile: ${candidate.linkedin}
- Current Location: ${candidate.location}
- Relocation Status: ${candidate.relocation}
- Work Authorization: ${candidate.workAuthorization}
- Availability: ${candidate.availability}
- Total Experience: ${candidate.experience}
- Expected Salary: ${candidate.expectedSalary}

Below are the details of the post I am applying to:
- LinkedIn Post URL: ${'postUrl'}

----------------------------------------
Job Post Description:
${jobDescription}
----------------------------------------

I would appreciate the chance to discuss how my experience aligns with your requirements. Thank you for your time and consideration.

Best regards,
${candidate.fullName}`;
```

```
369
370         // --------------------------------
371         // Send email
372         // --------------------------------
373
374         if (email) {
375
376             console.log(
377                 'Sending resume to:',
378                 email
379             );
380
381             await transporter.sendMail({
382                 from: process.env.EMAIL,
383                 to: email,
384
385                 subject:
386                     `${role} C2C Application - ${candidate.fullName}`,
387
388                 text: emailBody,
389
390                 attachments: [
391                     {
392                         filename: 'Resume.docx',
393                         path: './MyResume.docx'
394                     }
395                 ]
396             });
397
398             console.log(
399                 'Resume sent:',
400                 email
401             );
402
403         } else {
404
405             console.log(
406                 'No email found. Skipping.'
407             );
408         }
409     }
410 });
411
412
```